

EECE.5862 — Embedded Artificial Intelligence

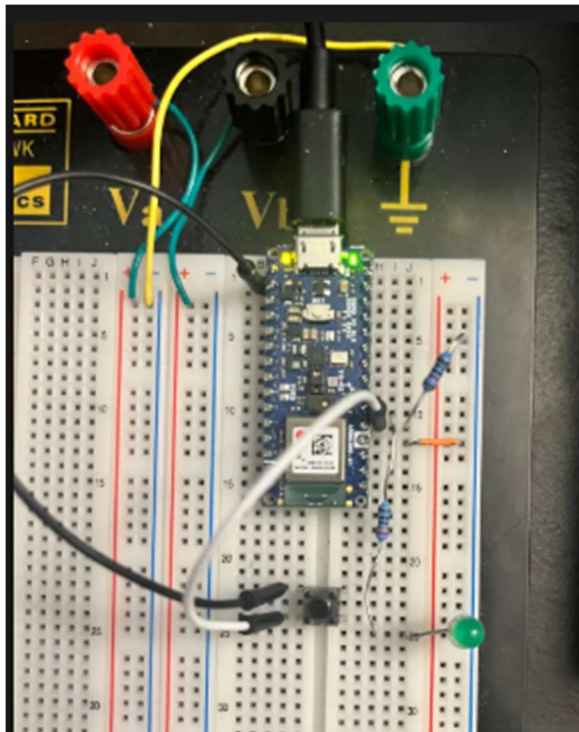
Lab 1 Report — Microcontroller Programming + First TFLite Micro Inference

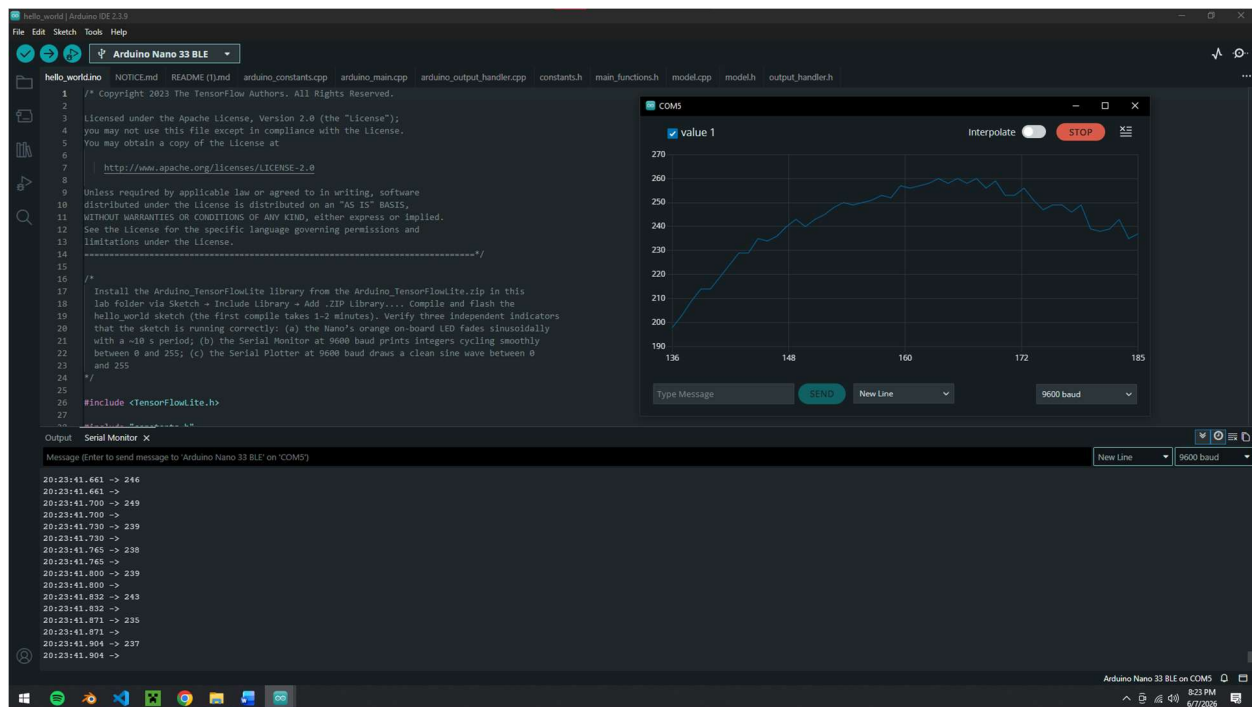
Name	Himadri Saha
Student ID	01951522
Course section	EECE.5862 (graduate)
Lab	Lab 1: Microcontroller Programming + First TFLite Micro Inference
Date submitted	2026-06-07
Instructor	Prof. Yan Luo

1. Objectives

Lab 1 was a refresher in core embedded skills (digital I/O, debouncing, interrupts), introduced TensorFlow Lite Micro, and explored how embedded ML models interact with the hardware real time. It characterized the model's interactions with the hardware, by verifying that the model's output can drive physical interfaces quantifying the recourse costs associated with running the model. Overall, lab 1 explored how resource constrained devices behave under real-time pressure.

2. Setup





- **D3** → one end of the **220 Ω resistor**
- Other end of the 220 Ω resistor → LED anode (long leg)
- LED cathode (short leg) → **GND**

3. Method and Results

Requirement 8.

Measurements: 347,880 bytes flash / 54,048 bytes RAM

Requirement 9.

Style	Workload	Worst-case response
Interrupt	delay(1)	1,050 μ s
Interrupt	delay(10)	10,752 μ s
Interrupt	delay(50)	50,023 μ s
Polling	delay(1)	19 μ s
Polling	delay(10)	27 μ s
Polling	delay(50)	8 μ s

3.1 Part A.1 — *Blink*

```
void req1(){
  digitalWrite(LED_BUILTIN, false);
  delay(1000);
  digitalWrite(LED_BUILTIN, true);
  delay(1000);
}
```

3.2 Part A.2 — *Blink + Serial logging*

```
void req2() {
  digitalWrite(LED_BUILTIN, true);
  Serial.println("LED ON @ " + String(millis()) + " ms");
  delay(1000);
  digitalWrite(LED_BUILTIN, false);
  Serial.println("LED OFF @ " + String(millis()) + " ms");
  delay(1000);
}
```

3.3 Part A.3 — *Polling-based debounced button*

```
void req3() {
  // Get current state
  unsigned long current_time = millis();
  bool current_button_state = digitalRead(BUTTON_PIN);

  // When the button is pressed, check if the button is debounced
  if (current_button_state) {
    if ((current_time - lastDebounceTime) > DEBOUNCE_DELAY) {
      // Toggle LED state
      ledState = !ledState;

      // Reset debounce timer
      lastDebounceTime = current_time;
    }
  }
}
```

3.4 Part A.4 — Interrupt-based debounced button

```
void buttonISR() {  
    // Check debounce  
    unsigned long current_time = micros();  
    if ((current_time - lastDebounceTime) > (DEBOUNCE_DELAY * 1000) ) {  
        ledState = !ledState;  
        lastDebounceTime = current_time;  
        buttonPressed = true; // Set flag for main loop  
    }  
}  
  
// Check if button was pressed and handle in main loop  
if (buttonPressed) {  
    Serial.println("Button pressed: ");  
    buttonPressed = false; // Clear flag  
}
```

3.6 Part B.1 — Annotate hello_world.ino

```
("Model Array: Maps existing g_model byte array to a useable tflite::Model* object");  
("AllocateTensors(): Reserves space inside tensor_arena for the model's input/output/intermediate tensors and reports whether they fit via allocate_status.");  
("Invoke(): Executes the model's forward computation on the current input tensor and writes its prediction into output, checking invoke_status for errors.");  
("Output post-processing: Dequantizes output's int8 value into a float y via its scale/zero_point, then hands (x, y) off to HandleOutput()");
```

3.7 Part B.2 — Compile & verify hello_world

3.8 Part B.3 — Instrument for inference latency

```
// Rolling-window latency accumulators (B.3): track Invoke() timings  
// across the current batch of inferences so min/mean/max can be  
// reported once the window fills, then reset for the next batch.  
constexpr int kLatencyWindowSize = 100;  
int latency_sample_count = 0;  
unsigned long latency_sum_us = 0;  
unsigned long latency_min_us = 0xFFFFFFFFUL;  
unsigned long latency_max_us = 0;
```

```

unsigned long invoke_start_us = micros();

// Run inference, and report any error
TfLiteStatus invoke_status = interpreter->Invoke();
if (invoke_status != kTfLiteOk) {
    MicroPrintf("Invoke failed on x: %f\n", static_cast<double>(x));
    return;
}

// Accumulate this Invoke() timing into the rolling window. Serial is
// used (not MicroPrintf) because this TFLM build's MicroPrintf mishandles
// %lu for the unsigned long that micros() returns.
unsigned long invoke_elapsed_us = micros() - invoke_start_us;
latency_sample_count++;
latency_sum_us += invoke_elapsed_us;
if (invoke_elapsed_us < latency_min_us) latency_min_us = invoke_elapsed_us;
if (invoke_elapsed_us > latency_max_us) latency_max_us = invoke_elapsed_us;

if (latency_sample_count >= kLatencyWindowSize) {
    unsigned long latency_mean_us = latency_sum_us / latency_sample_count;
    Serial.print("[LAT] min=");
    Serial.print(latency_min_us);
    Serial.print(" us, mean=");
    Serial.print(latency_mean_us);
    Serial.print(" us, max=");
    Serial.print(latency_max_us);
    Serial.println(" us");

    latency_sample_count = 0;
    latency_sum_us = 0;
    latency_min_us = 0xFFFFFFFFUL;
    latency_max_us = 0;
}

```


3.9 Part B.4 — Reflection paragraphs

3.10 Part C.1 — Polling vs. interrupt latency characterization

```
void recordPressResponse(unsigned long elapsed_us) {
    response_time_us = elapsed_us;
    press_count++;
    if (elapsed_us > worst_case_response_us) worst_case_response_us = elapsed_us;

    if (press_count >= kPressBatchSize) {
#ifdef USE_INTERRUPT
        const char* style_name = "interrupt";
#else
        const char* style_name = "polling";
#endif
        Serial.println("[RESP] worst-case = " + String(worst_case_response_us) + " us over "
            + String(press_count) + " presses (style=" + String(style_name)
            + ", workload=" + String(WORKLOAD_DELAY_MS) + "ms)");

        press_count = 0;
        worst_case_response_us = 0;
    }
}
```

4. Analysis

Requirement 8:

A) When measured against the Nano 33 BLE Sense's actual hardware limits — 1 MB (1,048,576 B) flash and 256 KB (262,144 B) SRAM, ~33% of flash and ~21% of RAM. The actual sine-wave model (g_model byte array) is only on the order of a couple KB. The bulk of the 347 KB is the TFLM interpreter, op resolver, and Mbed OS core, not the model itself. A 10× larger model (~20–30 KB): trivial — easily absorbed by the ~635 KB of flash headroom remaining. 50×–100× (~125–250 KB) larger model might fit, but RAM becomes a constraint first. TFLM needs to hold more tensors for every layers weights and activations at once. As that scales with the model (an image for example), it might blow past the 208KB SRAM making that the real constraint.

B) A regression example is an accelerometer-driven model maps a window of raw vibration samples which is a continuous output. A classification example is a wake-word detection system, as it maps short audio snippets which are one of a small, fixed set of discrete labels.

Requirement 9.

Polling Style: Worst-case response scales almost linearly with the workload delay. This follows directly from the code structure: the ISR stamps `press_timestamp_us` at the instant of the physical press (hardware interrupt latency on the nRF52840 is sub-microsecond), but the actual `digitalWrite` toggle happens later in `loop()`, only once the blocking `delay(WORKLOAD_DELAY_MS)` finishes and `buttonPressed` is re-checked.

Interrupt style: Worst-case responses show no meaningful relationship to the workload. This is because `press_timestamp_us` is stamped only after the press is already detected and debounced. The measured interval spans just the few microseconds between two `micros()` calls with a `digitalWrite()` in between.

5. AI Tool Disclosure

Tool	What you asked it (prompt summary)	What it produced	How you modified / verified it
<i>Claude</i>	<i>In req9, why is the code not printing the RESP log every time I press the button, even beyond the debounce time</i>	<i>It's gated by <code>kPressBatchSize</code> (50) — <code>recordPressResponse()</code> only calls <code>Serial.println("[RESP]...")</code> when <code>press_count >= kPressBatchSize</code> (req9.ino:42-54). The per-press feedback you do see every time is the separate "Button pressed (interrupt style) response = ..." line (req9.ino:91) — [RESP] only fires once every 50 registered presses, then resets the counter to 0.</i>	<i>Used the AI output to brainstorm a starting point – none of the listed issues were the problem</i>

6. References

1. Iodice, G. M. *TinyML Cookbook* (2nd ed.). Packt, 2023.
2. Arduino. *Nano 33 BLE Sense Rev2 documentation*. *add the specific pages you used*.